drones

MDPI

Article

A2C-LLM: An Actor-Critic-Enhanced Large Language Model for UAV Swarm Multi-Target Task Allocation

Jie Bao, Yuping Zhang *, Ronghao Zhang and Peng Zhang

School of Automation Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China; jie.bao@std.uestc.edu.cn (J.B.); 202511060919@std.uestc.edu.cn (R.Z.); 202512061016@std.uestc.edu.cn (P.Z.)
* Correspondence: zyp_002@uestc.edu.cn

Highlights

What are the main findings?

- A2C-LLM integrates large language models with Actor-Critic reinforcement learning, forming a closed training-inference loop for UAV swarm task allocation, significantly outperforming pure LLM and pure RL baselines in task completion rate and convergence stability.
- Single-step MDP with structured feasibility constraints enables scalable coordination for large-scale UAV swarms, where constrained decoding and feasibility projection ensure the executability of generated commands.

What are the implications of the main findings?

- Combining lightweight foundation models (only 1.5B parameters) with principled RL training provides a viable pathway for deploying AI systems in safety-critical robotic applications requiring both semantic reasoning and execution guarantees.
- The semantic-numerical interface resolves the tension between semantic flexibility and execution guarantees, bridging the gap between high-level task allocation and low-level executable commands in dynamic multi-agent environments, with strong scalability for future low-altitude economy deployments.

Abstract

In UAV swarm adversarial applications, multi-agent task allocation requires high-level reasoning and accurate decision-making in dynamic environments. Although large language models (LLMs) have shown strong performance in zero-shot reasoning, they cannot generate optimal allocation strategies without environmental objective feedback. To address this problem, we present A2C-LLM, an Actor-Critic-enhanced large language model for adversarial UAV swarm task allocation. Unlike traditional methods that adopt sequential tracking, we adopt a single-step decision process for macro allocation to improve the efficiency of immediate allocation. In A2C-LLM, the LLM serves as the Actor network to understand the adversarial environment and generate coordination strategies, while a lightweight neural network serves as the Critic network to estimate expected rewards and calculate TD advantage for fine-tuning. Experimental results demonstrate that A2C-LLM significantly outperforms traditional heuristic algorithms and pure LLM baselines in task completion rate and robustness across various adversarial scenarios, showcasing the potential of integrating reinforcement learning feedback with foundation models for autonomous aerial systems.

Check for updates

Academic Editor: Pablo Rodríguez-Gonzálvez

Received: 4 April 2026
Revised: 13 May 2026
Accepted: 18 May 2026
Published: 22 May 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

Keywords: UAV swarm; large language model; reinforcement learning; task allocation; adversarial coordination; multi-agent coordination; actor-critic

1. Introduction

A fundamental challenge in UAV swarm coordination—dynamic multi-agent task allocation where two opposing teams pursue conflicting objectives in an adversarial contest—is the semantic-numerical gap: top-level task objectives aimed at achieving victory must be decomposed and translated into low-level executable commands in real time [1]. However, existing approaches either rely on pure optimization or pure learning, but both suffer from the lack of semantic understanding and environmental feedback, leading to poor adaptability in dynamic competitive environments. This raises the core question: How can we bridge the semantic-numerical gap in UAV task allocation without sacrificing real-time performance and execution guarantees?

Conventional heuristics and optimization solvers suffer from severe processing delays as multi-agent networks scale, eliminating the real-time adaptability required for effective autonomous operation [2–4]. Meanwhile, large language models (LLMs) demonstrate strong performance in macro-strategy and zero-shot reasoning [5,6], but deploying vanilla LLMs in dynamic adversarial environments is risky. These models rely heavily on offline prompts without stable reinforcement learning feedback loops, preventing them from improving through environmental interaction [7]. Furthermore, standalone LLMs cannot guarantee executable outputs under rigid operational constraints [8–10].

To solve the above problems, we combined large language models and the Advantage Actor Critic (A2C) algorithm, and named this new hierarchical model A2C-LLM. The A2C-LLM model simplifies the problem's complexity by discarding conventional continuous control models and transforming the macro allocation problem into a simple Markov Decision Process (MDP). The Actor in our A2C-LLM is based on a 1.5 billion parameter model. After fine-tuning this model using specific scenarios and formatting, we found that this model learned to translate complex scenario conditions into clear coordination instructions. At the same time, we also employed a Critic model to generate value functions in real time. Having created this loop, we then feed the results from this single episode into our LLM. This single-step training process is a full cycle of reinforcement learning, covering everything from initial observation to policy updates. In this single-step setting, the advantage estimation simplifies to the temporal difference (TD) error $A(s, a) = r - V(s)$. Additionally, in order to stabilize our training in a non-stationary environment, we also employed KL regularization [11].

To assess policy stability along with the rates of task completion and perfect win rate, we conducted detailed simulations using representative multi-agent adversarial scenarios. We evaluated our framework against a variety of baselines, namely, REINFORCE-LLM [12], Hungarian algorithm [13], Random, Frozen-LLM, and PURE-A2C [14,15]. For ablation, we considered reinforced constraints [16] and fine-tuned LLM [17]. We evaluated our model for various critical aspects, namely, convergence, scalability, robustness, and its sensitivity towards different adversarial capabilities. We observed that our approach outperformed all other baselines for most of the settings. We observed faster convergence for our model and better scalability and robustness, even for stronger opponents.

To address this question, we propose A2C-LLM based on three key insights. First, single-step macro allocation reduces decision complexity while maintaining tactical coherence, enabling efficient high-level reasoning. Second, TD advantage with structured constraints allows reinforcement learning to operate with execution guarantees, ensuring

output feasibility. Third, LLM-in-the-loop RL leverages semantic prior knowledge for sample-efficient adaptation to dynamic environments.

These insights lead to four technical contributions. (1) We develop an embedded LLM simulation framework that integrates physics-based multi-agent simulation with reward feedback loops for continuous learning and validation. (2) We design a unified text decision interface with structured state templates that decouple high-level macro allocation from low-level reactive control, using constrained decoding and feasibility repair to bridge semantic reasoning and physical execution, and validate the effectiveness through experiments. (3) We propose the A2C-LLM architecture with strict single-step decision formulation, where the LLM serves as Actor paired with a lightweight Critic for value estimation, employing TD advantage learning and KL regularization for stable convergence. (4) We conduct systematic evaluation through multi-dimensional experiments and ablation studies, and the results demonstrate that this method exhibits superior robustness: a 25.7% relative improvement in perfect win rate over the second-ranked REINFORCE-LLM method, and a 9.8% relative improvement in task completion rate over the second-ranked Hungarian method.

2. Related Work

Multi-agent actor-critic methods have been developed for mixed cooperative–competitive environments [18]. Multi-objective task allocation has been modeled using MRTA or Dec-POMDPs formalisms [19,20], with major difficulties from non-stationarity and partial observability. Compared to optimization methods requiring accurate priors [21,22], RL enables cooperative decision-making through environment interaction [23].

A3C/A2C methods enhance policy improvement stability through parallel sampling and advantage estimation [24]. Barros employed SAC for UAV target tracking and obstacle avoidance [25], while Koch applied RL to attitude control and path planning [26]. Multi-agent Actor-Critic extensions have been used for UAV task allocation, cooperative tasks, and competitive contexts [19,20,27–29]. However, pure MARL methods remain limited by sample inefficiency and weak semantic prior utilization [30,31].

In addition to RL-based methods, bio-inspired swarm planning approaches have demonstrated effective coordination and task allocation for UAV swarms. Wang and Zhang [32] proposed a bionic wolf pack method that models the behavioral characteristics of wolf packs for dynamic task allocation with load balancing and coverage search. Xu et al. [33] developed a bio-inspired reinforcement learning framework that exploits collective milling structures and chiral dynamics to enable distributed active swarms to capture and merge targets. Fan et al. [34] combined pigeon flock leadership mechanisms with flocking algorithms for UAV swarm formation control and obstacle avoidance. In parallel, cross-domain collaborative methods have advanced multi-agent coordination across heterogeneous platforms. Dong et al. [35] proposed a hierarchical heterogeneous multi-agent framework that integrates UAVs, USVs, and UUVs via multi-policy reinforcement learning for underwater target searching. Hu et al. [36] developed a distributed guiding-vector-field controller for heterogeneous UAV-USV coordinated navigation with lightweight virtual-coordinate communication, validated through real-lake experiments. These developments in bio-inspired and cross-domain coordination provide a broader context for the present work, which explores the integration of LLM-based semantic reasoning into UAV swarm task allocation.

Recently, LLMs have shown promise in robotics and autonomous systems [37,38], providing global guidance for UAV swarm control [39,40]. However, their rapid adaptability in competitive environments remains limited. LLM-MARL integration has been explored: Zhu [41] for semantic region decomposition, Xu [42] for emergency networking with in-

trinsic rewards, and Zhou [43] combining LLM global understanding with Q-learning for mFSTSP. Unlike these approaches, we propose an LLM-in-the-loop RL framework where the LLM provides semantic task allocation via single-step decisions, with A2C architecture providing environmental feedback through a Critic network for incremental learning, and constrained decoding ensuring action executability.

3. Problem Formulation and System Framework

In this section, we first formalize the task allocation problem as a single-step Markov Decision Process (MDP). Then, we present the overall A2C-LLM framework that operates in a closed-loop manner. Finally, we describe the environment modeling, including the key entities and their interactions.

3.1. Single-Step MDP Formalization

We model the high-level allocation window as a single-step MDP (horizon 1). Each decision window has length ΔT. At the beginning of the window t, the agent receives the state s_t and outputs a mixed action a_t. The environment runs in a closed loop over $[t, t + \Delta T]$ and returns a scalar reward $r_t = r(s_t, a_t)$ at the end of the window, with the episode terminating immediately (done = 1). This setup is equivalent to a contextual bandit where cross-step effects are folded into the within-window reward distribution $p(r|s, a)$. The objective is to maximize the expected reward $J(\theta)$:

$$J(\theta) = \mathbb{E}_{s\sim\mathcal{D}, a\sim\pi_\theta(\cdot|s)}[r(s, a)] \tag{1}$$

where $\mathcal{D}$ is the window-level state distribution induced by the closed-loop simulator.

Data are collected in episode-level batches, with each episode containing exactly one single-step cycle. Due to the nature of our single-step decision process (horizon = 1, done = 1), the advantage estimation simplifies to the first-order temporal difference (TD) error:

$$A(s_t, a_t) = r_t - V_\phi(s_t) \tag{2}$$

This TD error serves as an advantage for policy gradient updates. The learnable baseline V_ϕ and in-batch advantage normalization help decrease gradient variance across different scenarios. KL regularization and constrained decoding with feasibility terms further stabilize training while allowing exploration, as evidenced by smoother learning curves in our experiments.

In contrast to the REINFORCE-LLM variants, where the importance-weighted $\log \pi_\theta(a|s)$ is used with Monte Carlo returns without a baseline, the use of the learnable baseline V_ϕ and the in-batch normalization helps to decrease the variance of the gradients for different scenarios. The KL regularization and the constrained decoding and feasibility terms also make the policy less susceptible to drift and output format problems while still allowing for exploration. The results from our experiments also show smoother learning curves. The insensitivity to reward weights and temperature is discussed from an architectural perspective in Section 5.6.

The choice of a single-step MDP formulation is grounded in a deliberate trade-off that substitutes temporal credit assignment with comprehensive state-space coverage.

In a conventional multi-step adversarial engagement, the decision-maker must sequentially react to evolving adversarial dynamics. Our approach re-conceptualizes this problem: instead of modeling a single longitudinal match, we treat each randomly generated episode as an independent sample from the distribution of all possible adversarial fragments—from opening-phase full-force deployments, through attritional mid-phase engagements, to endgame mop-up operations. By extensively randomizing red-blue force ratios, initial

position distributions, and red-team capability configurations during training, the model traverses the entire state space of the adversarial scenario. A model capable of making optimal macro-allocation decisions for any such fragment is functionally equivalent to a decision-maker that makes correct decisions at every step of a continuous timeline. One does not train a chess player to play every full match from start to finish; one trains an endgame master who can solve any board position.

This formulation carries an indispensable practical benefit: embedding the LLM in a full multi-step engagement—where inference and feedback would be required at every decision step—would cause per-episode training time to grow linearly with the number of steps, rendering large-scale iterative training computationally prohibitive. The single-step design compresses each episode into one independent contextual decision, making reinforcement learning with LLM-in-the-loop practically achievable without sacrificing representational completeness.

In summary, the single-step MDP is not a neglect of long-term planning, but an equivalent realization achieved by substituting temporal reasoning with state-space coverage under given computational constraints.

In the current framework, the decision window length ΔT is fixed across all training episodes. The specific value was determined through multiple trial runs to ensure that, in typical scenarios, the majority of capture and guard tasks can close within the window, thereby providing meaningful reward signals for the Critic. Adaptive adjustment of ΔT—for instance, triggering a new planning cycle dynamically based on situational changes—is discussed as a future direction in Section 7.

3.2. Overall A2C-LLM Framework

In this paper, we propose A2C-LLM, an episodic decision-making pipeline for task allocation in multi-target capture, with its architecture depicted in Figure 1. The system operates in a closed loop of state extraction, policy reasoning, command grounding, online feedback, and incremental adaptation.

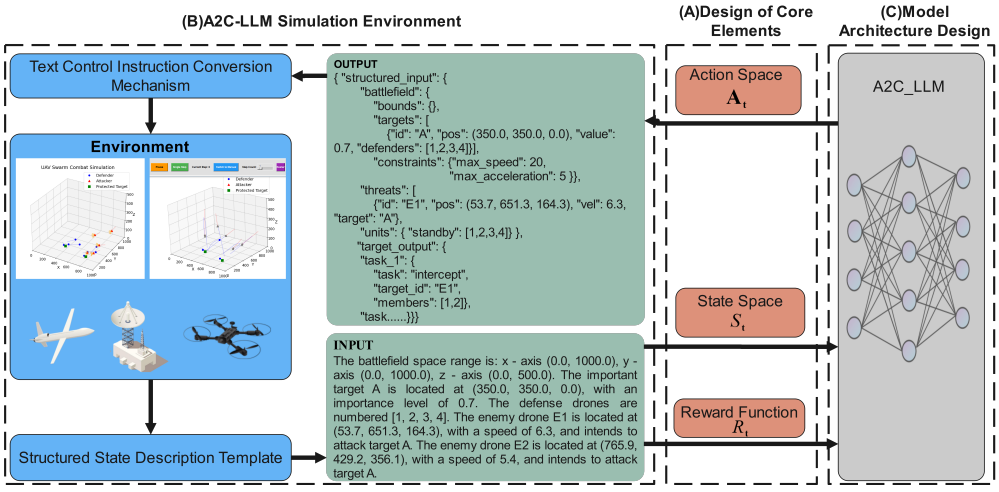


Figure 1. Overall architecture of A2C-LLM a dynamic closed-loop framework for multi-target capture task allocation.

The simulation environment employs a structured state template to extract information from multiple sources, including target sets, friendly platforms, Red Team Information, and environmental constraints. These situational elements are represented as standardized text-symbol hybrid inputs. The task allocation problem is formalized as an MDP with triplet (S_t, A_t, R_t), where S_t aggregates situational elements at time t, A_t represents cluster-level

tasking actions (capture/guard/standby), and R_t is a composite return including capture effectiveness, target loss, and format compliance.

The A2C-LLM agent consists of two main components. The LLM serves as the Actor, generating structured task directives that are format-correct, resource-feasible, and directly executable. The lightweight Critic estimates state values and provides TD advantage for policy updates, with KL regularization ensuring stability. LoRA enables efficient online adaptation by injecting environmental feedback into the policy head. Command grounding transforms high-level semantics into executable control directives, decoupled from low-level controllers. Catchers, i.e., the Blue Team, use proportional navigation (PN) [44] for tracking, while runners, i.e., the Red Team, employ artificial potential fields (APFs) [45] for evasion. This architecture forms a closed loop for stable and scalable cooperative decision-making.

3.3. Environment Modeling

In this study, we formulate a competitive capture contest scenario. The Blue Team serves as the catchers, while the Red Team acts as the runners. The objective of the Blue Team is to capture all Red Team UAVs, whereas the Red Team aims to reach the protected zones (scoring zones) before being captured. This setup creates a dynamic adversarial environment where both teams employ strategic maneuvers to achieve their respective goals.

In this study, air-based UAV coordination is formulated as a competitive multi-UAV task assignment and capture problem. A global right-handed Cartesian frame O-XYZ is used to describe the 3D workspace. The target set is $\mathcal{T} = \{T_i\}_{i=1}^{m}$, where the position of T_I is $p_i^T = (x_i^T, y_i^T, z_i^T)$. To encode target importance, a weight set $\mathcal{W} = \{w_i\}_{i=1}^{m}$ is assigned with $w_i \in [0,1]$ and $\sum_{i=1}^{m} w_i = 1$; if T_I is captured by the Red Team, the incurred score loss equals w_i. The catcher UAVs (Blue Team) form the set $\mathcal{R} = \{R_j\}_{j=1}^{n}$ with positions $p_j^R(t) = (x_j^R(t), y_j^R(t), z_j^R(t))$, and the runner UAVs (Red Team) form the set $\mathcal{B} = \{B_\ell\}_{\ell=1}^{k}$ with positions $p_\ell^B(t) = (x_\ell^B(t), y_\ell^B(t), z_\ell^B(t))$. The Euclidean distance between R_j and B_ℓ at time t is $d_{j\ell}(t) = \|p_j^R(t) - p_\ell^B(t)\|_2$. Adversarial coordination is realized through an assignment function $\phi : \mathcal{R} \to \{I, D, S\}$, where I denotes capture, D denotes guard, and S denotes standby. When $\phi(R_j) = I$, a specific target is designated via $\text{Capture}(R_j, B_\ell) : R_j \to B_\ell$, and R_j tracks and pursues B_ℓ under proportional navigation guidance to meet real-time requirements. When $\phi(R_j) = D$, a guard location and the scoring zone are specified via $\text{Guard}(R_j, P_d, T_i) : R_j \to (P_d, T_i)$, where $P_d = (x_d, y_d, z_d)$. The UAV R_j establishes a detection zone of radius $R_{\text{detect}} = 150$ m around P_d; once any runner enters this zone, i.e., $\min_{\ell} d_{j\ell}(t) \le R_{\text{detect}}$, the task switches automatically to capture, and R_j prioritizes the nearest opponent, $\ell^* = \operatorname{argmin}_{\ell} d_{j\ell}(t)$, then executes $\text{Capture}(R_j, B_{\ell^*})$. When $\phi(R_j) = S$, the UAV holds its current position or proceeds to a designated rally point until a new command is issued.

4. Semantic-Numerical Interface Design

To bridge the semantic-numerical gap, where LLMs excel in natural language while traditional A2C only accepts numerical inputs, this paper constructs a multimodal information encoding and instruction conversion mechanism. Through the structured state template, the original scene numerical values are normalized into a stable semantic text S_{text}, and a numerical vector S_{num} is generated simultaneously. Via the Hierarchical Conversion Bridge (Text Control Instruction Conversion Mechanism), the constrained JSON semantic instruction A_{text} output by the model and its matching parameter A_{num} are mapped to underlying control commands, achieving an executable and auditable one-step decision-making closed loop.

To suppress large-model hallucinations and ensure feasibility within the window, we introduce enhanced constraints and constrained decoding at both the prompting and decoding levels. Combined with feasibility repair and the corresponding penalty term R_{sf}, it thus supports the efficient training and stable deployment of A2C-LLM in the cooperation of UAV swarms, as illustrated in Figure 2.

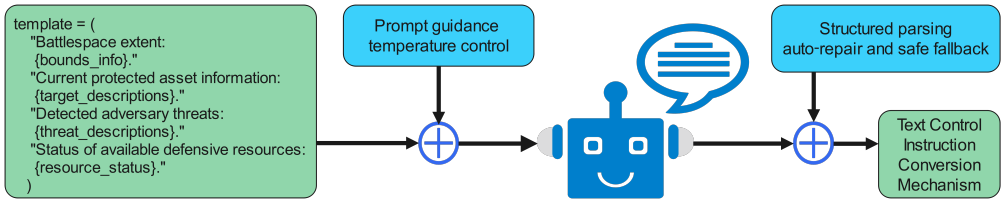


Figure 2. Multimodal information encoding and instruction conversion mechanism.

4.1. Structured State Template

To accommodate sensor integration in real-world applications, this study designs a hierarchically parameterized structured state template. It further refines the multi-modal operational information into four categories of standardized texts, namely:

1. Spatial Information, e.g., "Arena spatial range: x-axis$(x_{\min}, x_{\max})$, y-axis $(y_{\min}, y_{\max})$, z-axis $(z_{\min}, z_{\max})$";
2. Scoring Zone Encoding, e.g., "Zone A is located at (x,y,z), with importance p, and assigned guard UAVs: [n1,n2]";
3. Own UAV Status, e.g., "UAV D1 is located at (x,y,z), task status is X, and the set of executable actions is Y";
4. Red Team Information, e.g., "Red Team E1 is located at (x,y,z), speed is v, direction points to Zone B, and capability level is high".

The template constructs a logical framework of the four-layer structure, including spatial definition, target description, opponent situation, and resource status, and uniformly employs the Python parameterized format. The "format_scenario_description()" function is used to dynamically input real-time data, where the synchronous generation of semantic text S_{text} and numerical vector S_{num} can be realized. During the generation process, integrated vocabulary standardization, fixed field order, unit and dimension unification, numerical precision cropping, ID system, and resource allocation are adopted. This ensures the certainty from the same input to the same output, achieves the systematic transformation from complex environments into LLM-friendly expressions, and conforms to the one-step decision window in time.

The formal representation is

$$(S_{\text{text}}, S_{\text{num}}) = \mathcal{E}(x_{\text{raw}}; \Delta T) \tag{3}$$

where x_{raw} aggregates the original numerical stream, and $\mathcal{E}$ is a deterministic template encoder. In implementation, the format_scenario_description() function, while outputting S_{text}, returns the field-aligned S_{num}, and provides length truncation and outlier cropping strategies to ensure readability, interpretability, and stable support for downstream A2C-LLM training.

4.2. Text Control Instruction Conversion Mechanism

The instruction conversion pipeline consists of four sequential operations: parsing, canonicalization, feasibility projection, and control mapping. First, the parse function converts raw model outputs into structured candidate instructions. The canon function then performs key alignment and value range normalization, supplementing default fields and correcting non-standard values. The Π_F operator projects normalized instructions onto

the feasible set $F(s)$, performing minimal modification repairs when necessary. Finally, M maps the feasible instruction to the low-level control signal u. The unified conversion framework is expressed as:

$$\begin{aligned} u &= T(A_{\text{text}}, A_{\text{num}}; s, F(s)) \\ &= M\left(\prod_{F}(\text{canon}(\text{parse}(A_{\text{text}}), A_{\text{num}}))\right) \end{aligned} \tag{4}$$

When Π_F triggers a repair operation, the repair mark is recorded and included in the format penalty coefficient C_{format} calculation. For task_format inputs, the restricted JSON Schema validates, projects, and maps the instruction. The format discrimination rule is:

$$\text{format} = \begin{cases} \text{task_format}, & \exists \text{k} \in \text{keys}(\text{output}) \\ \text{json_format}, & \text{is_valid_json}(\text{output}) \\ \text{text_format}, & \text{otherwise} \end{cases} \tag{5}$$

For json_format inputs, key alignment is achieved through the field alias mapping table, with default parameter values supplemented according to Schema requirements. For text_format inputs, key information slot extraction and structured reconstruction are completed based on predefined rule templates and domain-restricted vocabularies. If the reconstruction result exceeds the feasibility constraint domain $F(s)$ or extraction fails, a high-weight C_{format} penalty is triggered, and the system falls back to a preset safe instruction (e.g., "maintain current attitude and standby state").

To ensure timeliness and traceability within the decision window, the conversion of $(A_{\text{text}}, A_{\text{num}})$ is performed immediately at the window start. This ensures the generated control instruction u is directly executable, and the operation process remains auditable. Algorithm 1 summarizes the instruction conversion pipeline, implementing function $\mathcal{T}$ from Equation (15). It takes raw model output, current state, and constraints as inputs, producing an executable command and violation flags for cost calculation (Equation (4)). The procedure includes format discrimination (Equation (3)), parsing, canonicalization, multi-criteria validation (schema, value ranges, logical conflicts), and entity resolution, culminating in the $\Pi_{\mathcal{F}}$ projection operation. If the instruction remains infeasible after repair, it falls back to a safe default.

Algorithm 1 Post-process-and-Project for Executable Command Generation

Require: Model output, state s, constraints $\mathcal{F}(s)$
Ensure: Executable command *cmd*, violation flags

```
fmt ← DetectFormat(output)
cmd ← Canonicalize(Parse(output,fmt))
flags ← {schema, range, conflict, unknown, repair, violation} ← 0
flags[schema] ← 0 if SchemaOK(cmd) else 1
cmd, e ← ClipRanges(cmd,s); flags[range] ∨= e
cmd, c ← ResolveConflicts(cmd,s); flags[conflict] ∨= c
cmd, u ← FixUnknownEntities(cmd,s); flags[unknown] ∨= u
cmd, r ← ProjectToF(cmd,F,s); flags[repair] ∨= r
if not ok then
    cmd ← SafeHold()
    flags[violation] ← 1
end if
return cmd, flags
```

4.3. Hallucination Mitigation and Feasibility Projection

In order to decrease the occurrence of structural hallucinations and OOB outputs without changing the timing of the one-step MDP, we propose the following single closed loop: constraint on the input side, repair on the output side, and an auditable record. At the beginning of the window, a controlled prompt will be generated according to the current state S. In this prompt, the Schema, key name whitelist, and range (ID list, speed, altitude, track, executable actions) are all specified. It is also necessary that "only entities already present are referenced from Stext, and no new IDs/fields shall be created" and that some minimal examples are provided to help the model understand the format and boundaries. Moreover, we allow the model to be exposed to a relatively low temperature and an appropriate upper limit on the length to help the model generate "only structured JSON instructions and avoid free text".

The constrained decoding mechanism operates through a five-layer post-processing cascade. First, the raw LLM output undergoes format detection to identify whether it conforms to the expected JSON schema, Python code block, or other structured formats. Second, direct JSON parsing is attempted for outputs beginning with valid JSON markers. Third, if the output is wrapped in Python code block markers, the inner dictionary structure is extracted and evaluated. Fourth, for partially corrupted outputs, brace matching is employed to extract any valid dictionary-like structures. Fifth, if all parsing attempts fail, a severe format penalty is applied, and the system falls back to a safe default action. Once successfully parsed, the output undergoes multi-stage validation, including schema verification, numerical range clipping for coordinates and velocities, logical consistency checks for mutually exclusive actions, and entity validity confirmation. For instructions violating feasibility constraints, minimal modification repair is applied through the projection operator Π_F. Unrepairable instructions trigger a fallback to the safe "Hold/Standby" command with associated violation costs. The entire process labels repairs and violation events and converts them into format cost items: let $f_{\text{schema}}, f_{\text{range}}, f_{\text{conflict}}, f_{\text{unknown}}, f_{\text{repair}} \in \{0,1\}$, then

$$\begin{aligned} C_{format} &= W_{schema} f_{schema} + W_{range} f_{range} + W_{conflict} f_{conflict} \\ &\quad + W_{unknown} f_{unknown} + W_{repair} f_{repair} \end{aligned} \tag{6}$$

Each weight is non-negative and clipped and normalized to the same magnitude scale as the reward function. If unrepairable, a high weight $C_{\text{violation}}$ is additionally imposed. To achieve traceability and comparability, the system saves the format branch, f_* flag, pre-and post-projection differences, rollback reasons, constraint version number, and random seed at each step. The field order and unit dimensions are fixed, and a stable algorithm is used for sorting. Thus, the complete closed loop of "front-end suppression $\rightarrow$ back-end repair $\rightarrow$ cost feedback" is embedded in Equations (15), (3) and (4).

It should be noted that the format-level constraints described above target only the structural well-formedness of the output, including schema completeness, member-assignment uniqueness, and red-team coverage feasibility. The model retains full degrees of freedom in all core decision dimensions—including but not limited to target selection for capture tasks, force allocation across multiple scoring zones, and switching among the three task types of capture, guard, and standby. None of these decisions is constrained by hand-crafted rules; they are optimized solely through task rewards returned by the environment.

We note that the structured state representation and the LLM's semantic reasoning capability are mutually constitutive in A2C-LLM: the state template translates raw numerical data into natural language precisely so that the LLM can exercise semantic understanding. PURE-A2C, operating on numerical vectors alone, cannot access this semantic dimension. The performance gap between the two should therefore be understood as arising from

the integrated effect of representation and reasoning, rather than from either component in isolation.

5. The A2C-LLM Architecture

5.1. Overall Architecture Design

The A2C-LLM architecture proposed in this paper (Figure 3), on the basis of maintaining the core advantages of the traditional Actor-Critic framework, deeply integrates the advanced reasoning ability of LLMs with the value estimation mechanism of reinforcement learning. The overall architecture consists of four core components: a policy network (Actor) with an LLM fine-tuned based on LoRA as the core, a value network (Critic) implemented using an independent multi-layer perceptron, a state encoder responsible for processing multi-source heterogeneous information, and a cooperative training mechanism that supports the joint optimization of heterogeneous components.

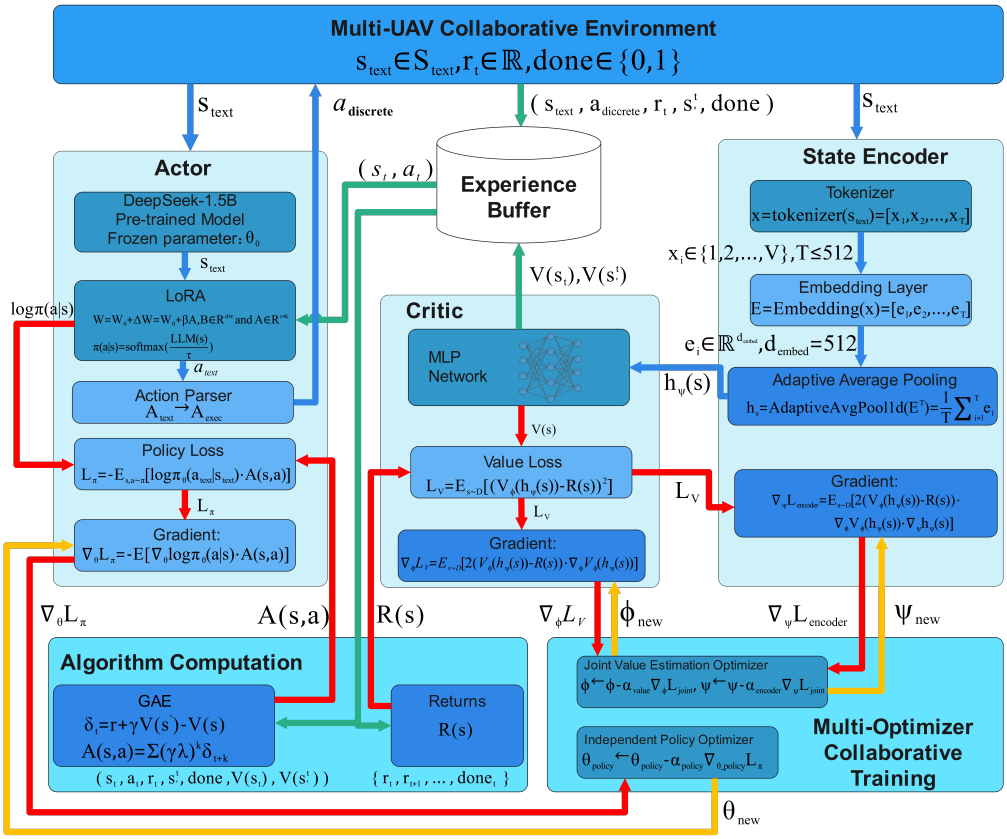


Figure 3. Architecture of the A2C-LLM Model. Among them, for the data and arrows: blue represents forward inference for decision-making, green represents data storage and algorithmic processing, red represents loss and gradient calculation, and yellow represents parameter optimization and update.

The core design concepts of this architecture are reflected in three aspects. First, by using a hybrid architecture mode, the complex reasoning ability of the pre-trained large model is combined with the mature advantages of traditional reinforcement learning algorithms in value estimation, thus ensuring the quality of decision-making while significantly reducing the complexity of end-to-end training. Second, a modular design strategy is adopted so that each component can be trained and optimized independently, effectively avoiding the numerical instability and computational complexity problems during the gradient propagation of the large model. Finally, in view of the characteristics of the UAV adversarial scenario, such as a long decision-making time window and a greater focus on episode-level global strategy optimization, a single-step decision-making mode

is adopted. At each decision-making moment, a complete coordinated allocation plan is directly generated, rather than a step-by-step action sequence, which is more in line with tactical cognition and actual needs. In this single-step setting, we use the TD advantage defined in Equation (2), while structured violations and repair events are converted into the format cost in Equation (14) and incorporated into the reward signal, forming a closed loop of policy generation, parsing and projection, execution, return feedback, and cost evaluation. A2C is adopted here as a simple and stable baseline; its combination with KL regularization provides update stability comparable to PPO's clipped objective while avoiding the latter's multi-epoch computational overhead.

5.2. State Encoder

Considering that the value network requires frequent calculations, a state encoder is adopted as a key component in the A2C-LLM architecture, serving as a bridge connecting the environment and the decision-making module. Through word embedding and multi-layer linear transformation, it compresses high-dimensional operational scenario texts into dense low-dimensional state vectors, providing a unified input for the value network. Its responsibility is to map the mixed state $s = (S_{\text{text}}, S_{\text{num}})$ into a fixed-length representation $h_\psi(s)$ for the value network to use; the action parsing and feasibility projection follow Equations (3), (4), and the subsequent reward functions. To maintain consistency with the policy branch, first, the tokenizer of the Actor is reused to map S_{text} into an embedding matrix $E \in \mathbb{R}^{T \times d_{\text{emb}}}$, and (adaptive) average pooling is used to obtain the text aggregation vector $h_{\text{text}} = \frac{1}{T}\sum_{i=1}^{T} e_i \in \mathbb{R}^{d_{\text{emb}}}$; then, the numerical features S_{num} are normalized by z-score and concatenated with h_{text}, and a fusion representation is obtained through a small MLP:

$$h_\psi(s) = MLP_\psi([h_{\text{text}}; \text{norm}(S_{\text{num}})]) \in \mathbb{R}^d \tag{7}$$

By default settings, the weights of the embedding layer and the base are kept frozen to control the scale of trainable parameters. The encoder and the value network are jointly trained end-to-end through the value loss. During training, gradients are not backpropagated to the policy branch, thus achieving branch decoupling and stable optimization. In this setting, the encoded representation $h_\psi(s)$ is directly used to estimate $V_\phi(s)$ and compute the advantage $A(s,a) = r - V_\phi(s)$. It should be emphasized that pooling and fusion only belong to the Encoder-to-Critic path and are independent of the generation computation graph of the Actor.

5.3. Design of LLM Policy Network (Actor)

Serving as the core network, the policy network introduces the pre-trained DeepSeek-R1-Distill-Qwen-1.5B as the base to realize the design of the LLM. During the entire model training process, only LoRA is used to perform efficient fine-tuning on it. The policy adopts the conventional autoregressive distribution modeling:

$$\pi_\theta(a|s) = \prod_{t=1}^{T} p_\theta(y_t | y_{<t}, s) \tag{8}$$

In the training phase of the model, to take into account both the stability and controllability of the language model in actual training, we adopt advantage weighting and incorporate the reference policy KL regularization and the entropy term:

$$\begin{aligned}\mathcal{L}_{actor} &= -\mathbb{E}_{s,a}[\log \pi_\theta(a|s)A(s,a)] \\ &\quad +\beta\mathbb{E}_s\Big[KL\Big(\pi_\theta(\cdot|s) \parallel \pi_{ref}(\cdot|s)\Big)\Big] \\ &\quad -\lambda\mathbb{E}_s[\mathcal{H}(\pi_\theta(\cdot|s))]\end{aligned} \tag{9}$$

Among them, π_{ref} is the frozen pre-training policy. The KL term is used to suppress the policy drift caused by online updates, and the entropy term is used to maintain an appropriate level of exploration. To avoid overfitting to the prompt segment, $\log\pi_\theta$, KL, and the entropy are calculated only in the generation segment, and teacher-forcing is adopted to evaluate the alignment probability.

5.4. Design of Value Network (Critic)

As an evaluation component in the A2C-LLM architecture, the value network is responsible for quantifying the value of the current operational situation. Unlike the policy network, which adopts the complex architecture of a large-scale language model, to avoid gradient coupling with the large language model, we implement the value network using a lightweight multi-layer perceptron (MLP). Since value estimation is essentially a regression task, compared with the complexity of policy generation, the lightweight architecture can significantly reduce training costs. Eventually, the total number of parameters of this network can be as low as $(512 \times 512 + 512 \times 512 + 512 \times 1) \approx 0.525$ M. Compared with the number of parameters of the base LLM in the policy network (about 1.5B), this approach enables the value network to quickly respond to policy updates and provide timely value feedback. Since the state encoder compresses scenarios of any scale into a fixed-length vector and the Critic only needs to estimate a single scalar value, this lightweight architecture provides sufficient capacity for the task. The experimental results further corroborate this design, where A2C-LLM maintains stable performance at larger scales.

Serving as the baseline for advantage calculation, the input of the value network is the fused representation $h_\psi(s)$ output by the state encoder. After being mapped through a two-layer fully connected network with ReLU activation, the state value $V_\phi(s)$ is obtained. The encoder parameters ψ and the value network parameters ϕ are jointly optimized by minimizing the mean squared error between the predicted value and the observed reward. In our MDP formulation, the return reduces to the immediate reward r, yielding the value loss:

$$\mathcal{L}_V(\phi,\psi) = \mathbb{E}_{s\sim\mathcal{D}}\Big[\big(V_\phi(h_\psi(s)) - r\big)^2\Big] \tag{10}$$

During training, only $\mathcal{L}_V$ is used to update ϕ and ψ, while isolating their gradients from the LoRA parameters of the Actor. The advantage $A(s,a) = r - V_\phi(s)$, after being standardized within each batch, is used for the Actor's update, and the gradient backpropagation to the Critic is stopped to avoid mutual interference.

5.5. Inter-Network Coordination and Optimization Mechanism

The inter-network coordination mechanism enables collaborative updates of the policy network, value network, and state encoder while ensuring stable end-to-end training under this MDP setup. We introduce a multi-optimizer approach with branch decoupling: the policy branch updates only the LoRA parameters θ, while the value branch jointly optimizes the lightweight MLP parameters ϕ and the encoder parameters ψ. Training follows a value-priority approach where, for each batch, the encoder first computes $h_\psi(s)$ and the Critic estimates $\hat{V} = V_\phi(h_\psi(s))$ without gradient flow to the Actor. The value loss drives the joint optimization of ϕ and ψ, while the Actor loss updates θ using the

standardized advantage $A(s,a) = r - V_\phi(s)$. This design ensures that gradients do not cross between branches, maintaining training stability.

5.6. Reward Design

The reward function employs a multi-level design, integrating task effects, stability, and format constraints.

The approach of this paper is a decoupling approach of upper-layer text representation and lower-layer numerical execution. The state space is defined as a structured scene mixed-state text $s_t = (s_{\text{text}}, s_{\text{num}})$ on a single path and contains information such as operational boundaries, attributes of targets and incoming aircraft, opponent situation assessment, formation/binding relationships, historical assignments, and key timings. The information is then structured into parseable text in a fixed template form and then tokenized and embedded into a fixed-length form.

The action space represents high-level mixed actions $a_t = (A_{\text{text}}, A_{\text{num}})$ with a strict JSON schema. Based on elements such as 'task type, participating units, action objects, and key positions', it defines core fields for tasks, members, target1_id, capture_point, and waypoints. After parsing and verification by the executor, it is transformed into numerical commands executable by the environment for task accomplishment, e.g., capture, guard, and patrol. However, it is worth noting that the upper layer only sends position/path parameter values. Key parameters of the controller, e.g., PN navigation constants, maximum acceleration, and safety radius, are uniformly set by the environment and not included in the high-level actions.

The reward function employs a multi-level design, integrating task effects, stability, and format constraints. Table 1 summarizes the key symbols used in this section.

Table 1. Key symbols for reward function and state-action interface.

Symbol	Definition
$\mathcal{T}$	Set of high-value targets
$\mathcal{R}$	Set of catcher UAVs (Blue Team)
$\mathcal{A}$	Set of runner UAVs (Red Team)
$\mathcal{A}_{\text{int}}$	Subset of successfully captured runners
$\mathcal{D} \subseteq \mathcal{T}$	Set of captured scoring zones
$w_i \in [0,1]$	Importance weight of target T_i
$w_{\max}$	$\max_i w_i$, normalization benchmark
η_{int}	Task completion rate, $\lvert\mathcal{A}_{\text{int}}\rvert / \lvert\mathcal{A}\rvert$
c_{int}	Capture reward coefficient (default 15)
P_{loss}	Target loss penalty (default -15)
B_{guard}	Perfect guard bonus (default $+15$)
R_{sf}	Format and stability penalty
$s_t = (s_{\text{text}}, s_{\text{num}})$	Mixed state representation
$a_t = (A_{\text{text}}, A_{\text{num}})$	Mixed action representation

The real task completion rate is defined as:

$$\eta_{\text{int}} = \frac{|\mathcal{A}_{\text{int}}|}{|\mathcal{A}|} \in [0,1] \tag{11}$$

where $\mathcal{A}$ denotes the set of all runner UAVs (Red Team), and $\mathcal{A}_{\text{int}} \subseteq \mathcal{A}$ represents the subset of runners that are successfully captured. The capture reward scales linearly:

$$R_{\text{int}} = c_{\text{int}} \cdot \eta_{\text{int}} \tag{12}$$

with default $c_{\text{int}} = 15$. For zone guard, let $\mathcal{D} \subseteq \mathcal{T}$ denote the set of captured scoring zones. The target protection reward is:

$$R_{\text{target}} = \begin{cases} -|P_{\text{loss}}| \cdot \frac{\sum_{T_i \in \mathcal{D}} w_i}{w_{\max}}, & \text{if } \mathcal{D} \neq \emptyset \\ +B_{\text{protect}}, & \text{otherwise} \end{cases} \tag{13}$$

where w_i is the importance weight of target T_i, and $w_{\max} = \max_i w_i$. Default values: $P_{\text{loss}} = -15$, $B_{\text{protect}} = +15$. The format and stability penalty aggregates all constraint violations:

$$R_{\text{sf}} = \sum_{v \in \mathcal{V}} p_v \tag{14}$$

where $\mathcal{V}$ is the set of violation types (parsing failure: -80, illegal action: -8, missing fields: -15 to -25, etc.). The total reward is:

$$R = R_{\text{int}} + R_{\text{target}} + R_{\text{sf}} \tag{15}$$

On the reduced sensitivity to reward weights. The stability observed across scenarios with a single set of reward weights can be understood through three architectural properties. First, the additive structure Equation (15) ensures each component contributes independently, avoiding nonlinear interactions when individual coefficients are scaled. Second, the KL regularization term in Equation (9) constrains policy drift under varying gradient magnitudes. Third, constrained decoding provides a hard feasibility boundary that prevents format violations from materializing, rendering penalty terms largely invariant to their specific numerical values. A systematic empirical ablation is deferred to future work.

5.7. Training Process

This section presents the end-to-end training process under this decision window (Algorithm 2). First, the input state is encoded. The Actor generates candidate outputs under a unified template, which are then parsed and subjected to feasibility checks to be converted into actionable moves, with corresponding reward feedback obtained. Subsequently, this feedback is used to construct advantages, and a parameter update is completed following the sequence of "updating the value first, then the policy". On the policy side, logarithmic probabilities and regularization terms are calculated solely based on the generated content; the parsing and feasibility check modules do not propagate gradients backward.

Algorithm 2 A2C-LLM Training Loop

for batch $\mathcal{D}$ **do**
  for state $s \in \mathcal{D}$ **do**
    $h \leftarrow \text{Enc}(s,\psi)$
    $(y, l_p) \leftarrow$ Actor.gen(s,θ,constrained=True)
    $a \leftarrow \text{Parse}(y)$
    $(r, \text{ok}) \leftarrow \text{Env}(a)$
    $V \leftarrow \text{Critic}(h,\phi)$
    $A \leftarrow \text{norm}(\text{detach}(r - V), \text{mask=ok})$
    $\mathcal{L}_V \leftarrow \text{MSE}(V[\text{ok}], r[\text{ok}])$
    Update $\{\phi, \psi\}$ using $\mathcal{L}_V$
    $\ell\ell \leftarrow \text{mean}(\text{logprob}(l_p), \text{mask="gen"})$
    kl $\leftarrow$ KLLogProb(logprob(s,y), mask="gen")
    $\mathcal{L}_\pi \leftarrow$ actor_loss($\ell\ell$,A,kl)
    Update $\{\theta\}$ using $\mathcal{L}_\pi$
  end for
end for

6. Experiments and Results

6.1. Experimental Setup

The experiments in this chapter are carried out based on a unified self-developed simulation platform and a set of predefined evaluation metrics. The comparative algorithms are diverse and span a wide range of approaches, from heuristic-based ones to those based on deep learning technology and from model-free ones to those based on large models. The opponent capabilities are classified along two aspects: intrinsic performance and the ability to avoid obstacles. The experiments are carried out based on a shared set of random seeds. The average values and the 95% confidence interval of four key metrics are adopted in the experiments. The entire environmental configurations, the classification of capabilities, and the comparative and ablation experiments are presented in Table 2 (setup and evaluation protocol overview, panels A–D).

Table 2. Experimental setup and evaluation protocol.

Panel A: Environment and Platform					
Module	**Configuration**				
Software/Environment	Python 3.8; PyTorch 2.0.0; 3D Cartesian world (1 km^3); velocity/acceleration/angular-rate constraints.				
UAV Parameters	Blue Team UAV: $v_{max} = 15$ m/s, $a_{max} = 2$ m/s^2; Red Team UAV: varies with capability level.				
Model	DeepSeek-R1-Distill-Qwen-1.5B; LoRA ($r = 16$, $\alpha = 32$, dropout = 0.1); 7 target modules.				
Hardware	Intel i7-14700KF; 32 GB RAM; NVIDIA RTX 4070 Ti.				
Training	LR 2×10^{-5}, batch size 2, update freq 2 episodes, 1600 episodes.				
Panel B: Adversary Capability Tiers					
Tier	**Capture Radius**	**Maneuver Mult.**	**Turn-rate Mult.**	**Perception Range**	**Repulsion Range**
Weak	10	1.0	0.5	0	0
Medium	5	1.25	0.8	100	30
Strong	3	2.0	1.2	200	80
Panel C: Baselines and Ablations					
ID	**Method**	**Description**			
A	A2C_LLM	Proposed: LLM fine-tuning + A2C with single-step advantage.			
B	A2C_FineTune	LLM fine-tuning only, no RL.			
C	Pure_A2C	Vanilla A2C with numeric state input.			
D	REINFORCE_LLM	LLM + REINFORCE.			
E	Hungarian	Rule-based assignment (Hungarian algorithm).			
F	Random	Random policy (lower bound).			
G	Frozen_LLM	Frozen LLM (no fine-tuning) + rule constraints.			
Panel D: Metrics and Evaluation Protocol					
Symbol	**Name**	**Definition**	**Reporting**		
R_{int}	Task completion rate (%)	Captured / total runners; averaged.	Mean [95% CI]		
$R_{perfect}$	Perfect-win rate (%)	Fraction of zero-loss episodes.	Same		
R_{hy}	Key-zone guard (%)	Weighted sum of guarded zones.	Same		
R_{comp}	Completion rate (%)	Parseable and executable outputs.	Same		

Note: Results use 100 random seeds; 95% CIs by bootstrap; paired comparison design.

6.2. Basic Performance Experiments

The basic performance of each method was initially evaluated in a normal environment. Unless otherwise stated, all experiments in this chapter were carried out in accordance with Table 2. The scenario was set as "12v7" and the opponent capability was set to "low" in the experiments. The basic group contains a range of approaches from rule-based ones to DRL and those based on large models. The experimental results show that A2C-LLM has initially proven its effectiveness. It not only has the capacity to efficiently detect and capture opponents but also has good stability in guarding scoring zones. As can be seen in the

confidence interval column in Table 3, the confidence interval of A2C-LLM does not overlap with those of other algorithms. This shows a significant advantage in its performance. Other strategies, such as RANDOM, a random-based strategy, are almost useless in guard capabilities. At the same time, the overall performance of PURE-A2C was not good. This again emphasizes the importance of choosing a good strategy for the agent. Although Hungarian achieved a relatively high value in each of the metrics, its confidence interval was still overlapping. This means that its performance may fluctuate greatly in different environments. Adopting the 12v7 configuration as the controlled baseline ensures fair comparisons among different methods. Subsequent sections present experimental results scaled up to 40v30, and further verify the model's robustness under random combined configurations of scale, capability and speed.

Table 3. Basic performance results (12v7 scenario, weak enemy).

Metric	A2C-LLM	REINFORCE	FineTune	Hungarian	Random	Frozen	PURE-A2C
R_{int} (%)	98.00 [96.9, 99.1]	86.57 [83.0, 90.2]	87.00 [83.0, 91.0]	94.57 [93.1, 96.0]	56.86 [52.4, 61.3]	18.57 [11.1, 26.0]	60.96 [57.5, 64.4]
R_{key} (%)	97.20 [95.6, 98.8]	83.80 [79.2, 88.4]	83.20 [78.1, 88.4]	92.40 [90.4, 94.4]	29.00 [23.4, 34.6]	64.70 [55.5, 73.9]	35.10 [27.9, 42.3]
R_{perfect} (%)	87.00 [80.3, 93.7]	43.00 [33.1, 52.9]	49.00 [39.0, 59.0]	63.00 [53.4, 72.6]	1.00 [−1.0, 3.0]	11.00 [4.8, 17.2]	3.00 [−0.4, 6.4]
R_{comp} (%)	100.00	100.00	99.00	100.00	100.00	57.00	100.00

Note: Results show mean [95% CI] from 100 random seeds. A2C-LLM shows non-overlapping confidence intervals versus all baselines.

6.3. Scenario Complexity Scaling Experiments

To characterize the impact of increasing scale on performance and identify inflection points, we fixed the opponent capability level and other scenario parameters, and only scaled the number of Blue Team/Red Team UAVs proportionally. For all methods, the training/inference decoding budgets and hyperparameters were kept consistent except for the scenario scale.

As illustrated in Figure 4, with the gradual expansion of scenario scale across five levels (8v6, 16v12, 24v18, 32v24, and 40v30), the task completion rate of A2C-LLM remains above 80% at all scales. In contrast, the performance of REINFORCE-LLM drops sharply as the scale increases, declining by nearly 50% from 16v12 to 40v30. The poor performance of PURE-A2C is mainly attributed to the fact that this model was only trained in the 12v7 scenario. When applied to more complex test environments, it fails to adapt to the new dimensions, thus being unable to maintain the desired operational effectiveness.

The experimental results show that our A2C-LLM approach can take full advantage of the generality of LLMs and the strength of A2C in RL. This can help our approach to tackle complex scenarios in a stable way, thus maintaining efficient allocation and improving the task completion rate. When confronted with complex scenarios, the REINFORCE approach cannot enjoy the advantage of A2C in evaluating the values, which may cause instability and performance deterioration in complex scenarios. The poor performance of the PURE-A2C approach indicates the dimensional mismatch between the training environment and the test environment. This again shows the effectiveness of using the idea of combining reinforcement learning and LLMs in addressing the problem of adaptability.

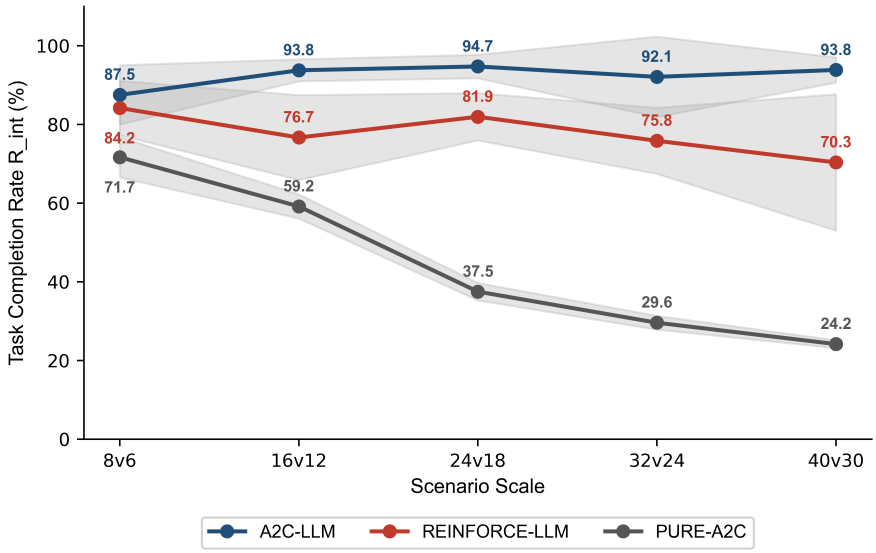


Figure 4. Task completion rate across scenario scales (mean ±95% CI). The gray shaded area indicates the 95% confidence interval.

6.4. Sensitivity Experiment on Opponent Capabilities

To validate the sensitivity of the proposed approach with respect to the changes in the capability of the opponent, experiments were conducted with a fixed density of 12v7, adjusting the parameters of the opponents to match the capability as depicted in Table 2. The experiments were designed to validate the extent to which the performance of each method deteriorates as the capability of the opponents improves, as well as the extent to which the relative performance of each method changes as this happens. Regarding the control logic, the Blue Team UAVs use the proportional navigation method for capturing the opponents, while the Red Team UAVs use the potential field method for evading the opponents. Therefore, the low-level maneuver effectiveness is not directly affected by changes in opponent speed and capability. Therefore, the primary aim of the experiments was to validate the effectiveness of the proposed method, i.e., A2C-LLM, as it adapts and becomes more effective at task allocation with respect to the changes in the capability of the enemies.

Based on the results depicted in Figure 5, it can be noted that the proposed method, i.e., A2C-LLM, adapts well with respect to the changes in the capability of the opponents and becomes more effective at task allocation, as depicted by the high rate of captures. It can also be noted from the results that the REINFORCE-LLM and PURE-A2C methods perform poorly as the capability and speed of the opponents increase, and fail to capture the opponents. The Hungarian method performs well for scenarios with low and medium difficulty levels, but performs poorly as the difficulty level increases. The Average Relative Decline Percentage (ARDP) values, as depicted in the figure, also validate the effectiveness of each method with respect to the changes in the capability of the opponents. The proposed method, i.e., A2C-LLM, performs the best with respect to stability and adaptability for changing scenarios with an ARDP of 11.61%, while the Hungarian method performs better than the others with an ARDP of 17.41%, which indicates fair stability and adaptability. These results further verify the importance of integrating large models in reinforcement learning, particularly when addressing tasks that require adaptation to dynamic changes.

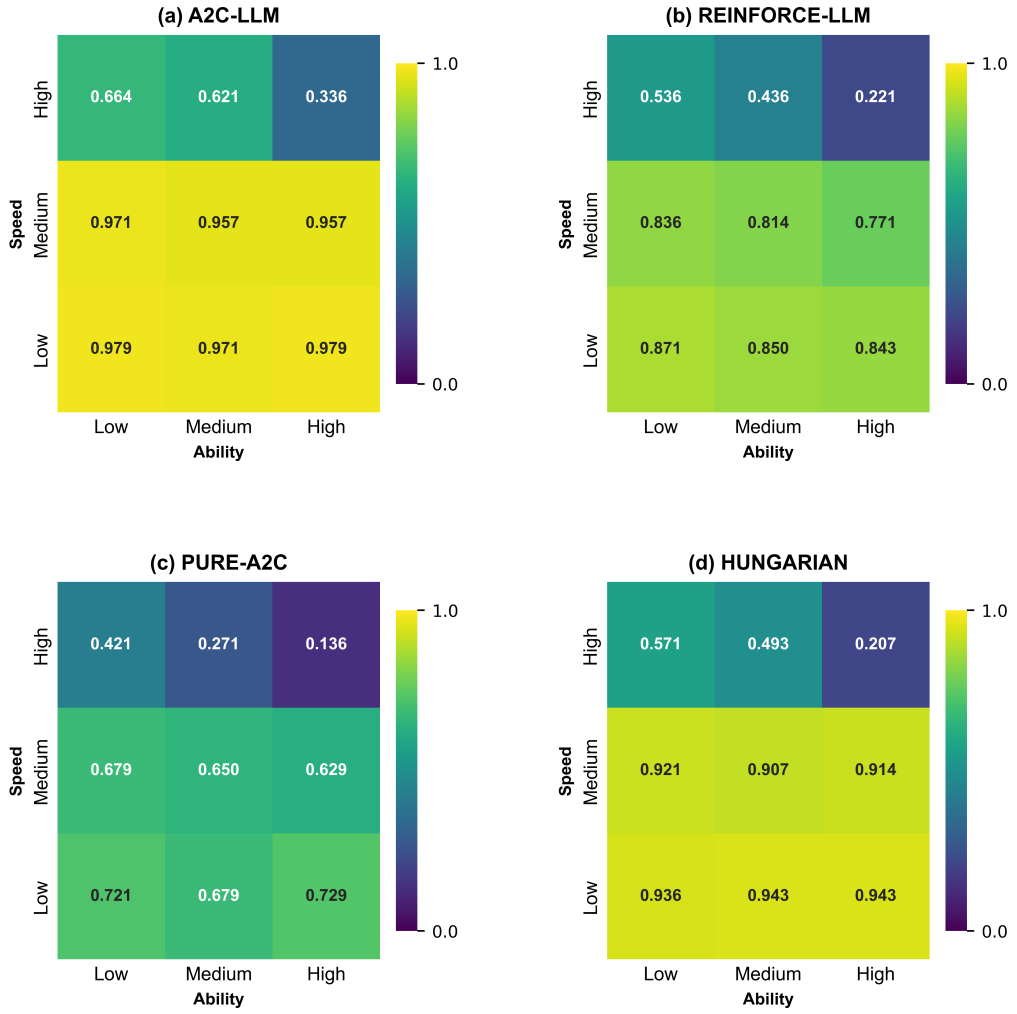


Figure 5. Sensitivity to opponent capability on a 3 × 3 grid of opponent_speed × opponent_ability. Color denotes mean task completion rate R_{int} (%) with a shared 0–100% scale; models are evaluated under identical settings (learning methods trained at the Standard level).

6.5. Multi-Dimensional Robustness Verification

The multi-dimensional robustness verification experiment is intended for verifying how each of the algorithms resists extreme uncertainty. In this case, the UAV swarm coordination system is subject to a highly unpredictable and intricate environment. At the beginning of each round, we randomly set up the scale, capability, and speed configurations and then test all of the algorithms on the same 100 random rounds (paired setting).

The results of this experiment are shown in Figure 6 and demonstrate that the application of the A2C-LLM approach provides robust and flexible decision-making. This is because it optimizes the policy and the value function simultaneously and therefore reduces variance and makes policy updates more stable and efficient in adapting to dynamic and intricate environments. The application of LLM improves the algorithm's capacity to understand complex information and its flexibility in adapting to diverse and dynamic situations in the environment.

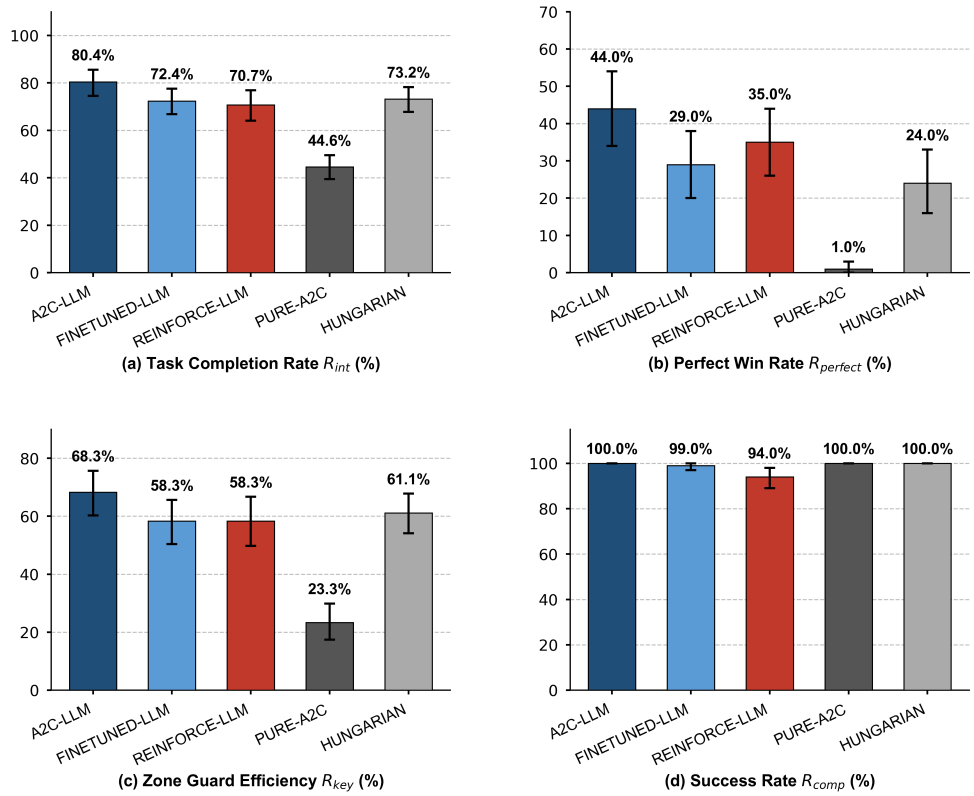


Figure 6. Baseline algorithm performance on 100 random episodes. Bars show sample means over 100 paired random episodes where scale, ability, and speed are resampled each episode. Error bars denote 95% confidence intervals via episode-level bootstrap (B = 5000).

6.6. Analysis Experiment on Learning Convergence and Efficiency

This study aims to examine the convergence of learning and the efficiency of the proposed reinforcement learning-based A2C-LLM in UAV swarm coordination scenarios. In this study, binning with light smoothing is implemented with a smoothing parameter set to 0.85 and bin_size set to 20, meaning we will average the results every 20 episodes.

If we examine the general trend in Figure 7, A2C-LLM exhibits the best convergence behavior, entering rapid convergence at episode 350 and stabilizing around episode 400. Its final task completion rate is 0.90 ± 0.15, outperforming REINFORCE-LLM (0.78 ± 0.38) by 13.79%. PURE-A2C, despite using the same A2C architecture, shows minimal improvement even after 16,000 episodes, with task completion rate increasing only 0.03 from the initial 0.58. This demonstrates that PURE-A2C, relying on numerical vector inputs, lacks the state representation enabled by LLM-based semantic understanding, and therefore fails to adapt to the dynamic adversarial scenario, unable to learn effective coordination strategies even with extensive training. In contrast, A2C-LLM achieves superior sample efficiency: converging in 1600 episodes what PURE-A2C fails to achieve in 16,000. The variance of A2C-LLM (0.0222) is also much lower than REINFORCE-LLM (0.1406), verifying better training stability.

The learning curve of A2C-LLM shows two-stage characteristics: accelerated convergence (episodes 1–400) and stable optimization (episodes 401–1600). During accelerated convergence, the pre-trained language model provides prior knowledge enabling rapid strategy optimization through environmental feedback. During stable optimization, performance converges with continuous fine-tuning. PURE-A2C shows flat learning curves even after 16,000 episodes, completely failing to adapt to the scenario changes and lacking effective convergence due to the absence of LLM-based semantic understanding.

REINFORCE-LLM benefits from semantic understanding but exhibits high variance and insufficient stability due to the algorithm's inherent characteristics.

Based on the systematic research on learning convergence and efficiency analysis, the excellence of the A2C-LLM method in the learning process of reinforcement learning is verified completely. It can be seen from the experimental results that not only does the introduction of LLMs improve the performance of decision-making in the end, but more importantly, it fundamentally improves the learning mechanism of the algorithm, which can significantly improve the speed of convergence and the efficiency and stability of the learning process.

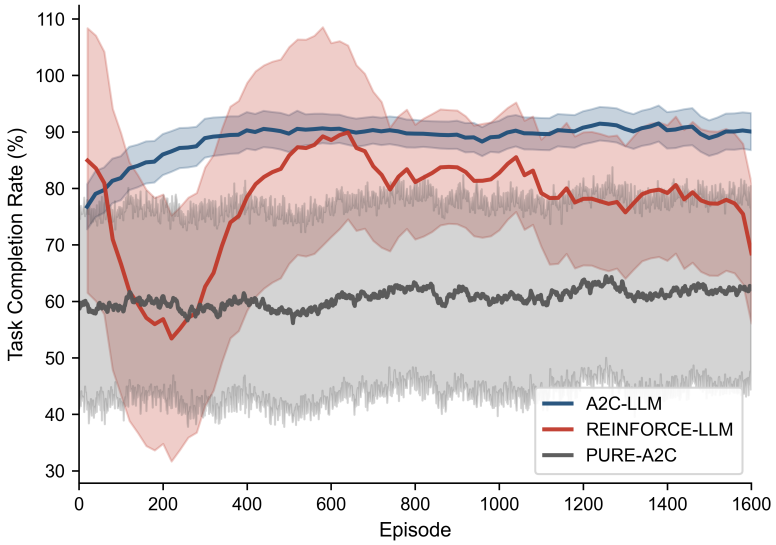


Figure 7. Reward convergence curves during training. A2C-LLM and REINFORCE-LLM trained for 1600 episodes; PURE-A2C trained for 16,000 episodes (mapped to 0–1600 interval for comparison). Binning and smoothing applied (smooth = 0.85, bin-size = 20).

6.7. Ablation Study

To evaluate the contributions of key components, we conduct ablation studies on two core modules introduced in Section 4.3: AugPrompt (controlled prompt generation with task instructions and format constraints) and AugParse (output format validation and feasibility projection with penalty feedback). AugPrompt corresponds to the structured state templates and constrained decoding mechanisms that provide semantic prior knowledge to the LLM, while AugParse corresponds to the post-processing pipeline that validates output feasibility and applies format penalties. The ablation groups include Full (= A2C − LLM), −AugPrompt, and −AugParse. The standard 24v16 environment (24 Blue Team UAVs vs. 16 Red Team UAVs) is adopted, with opponent capability and speed configured at medium levels.

Table 4 shows that Both Enabled achieves the highest task completion rate (93.62%) and perfect command compliance (100%), while AugParse Only yields a slightly higher perfect-win rate (36.0% vs. 34.0%). The two configurations are complementary: AugParse Only delivers marginally better zero-loss performance, whereas Both Enabled eliminates unexecutable outputs entirely. We select Both Enabled as the full configuration because, in deployment, even a single parse failure forces a fallback to the safe standby command, which can trigger cascading failures—a risk that outweighs a 2% difference in perfect-win rate. The 0.33 percentage point difference in task completion rate between Both Enabled and AugParse Only falls within the 95% confidence interval and is therefore not statistically significant. Notably, the deployment-critical value of full command compliance is further corroborated by the stress-test findings in Section 6.10: under strategy-level adversary

variation, A2C-LLM maintains a 52.0% perfect-win rate—outperforming the Hungarian baseline by 42.0 percentage points—yet still exhibits a 4% risk of catastrophic failure (zero captures). This observation reinforces the importance of eliminating parse-triggered fallback entirely, as even a single unexecutable output can cascade into mission loss.

Table 4. Ablation study results (24v16 scenario, medium opponent).

Group	R_{int} (%)	R_{key} (%)	R_{perfect} (%)	R_{comp} (%)
No AugPrompt and No AugParse	88.62	65.8	22.0	99.0
AugPrompt Only	65.25	34.3	7.0	100.0
AugParse Only	93.31	81.3	36.0	99.0
Both Enabled (Full)	93.62	80.1	34.0	100.0

Note: Ablation on prompt augmentation (AugPrompt) and parsing augmentation (AugParse). Full configuration achieves best balance of performance and stability.

6.8. Case Studies and Qualitative Analysis

Numbers alone cannot fully capture A2C-LLM's underlying allocation logic. We pulled two distinct simulation scenarios directly from our multi-dimensional robustness tests to show exactly how the system behaves. We map these critical capture assignments using 3D process visualization.

(1) Case Scenario 1: Analysis of a Complete Success Case

Figure 8a plots the initial approaching runners. Red Team UAVs E2 and E3 immediately approach our highest-value zone (Target A, importance weight 0.5). We positioned Blue Team UAVs 1 through 5 around this exact location. Concurrently, opposing units E4, E5, and E6 move toward Target D, a strictly lower-priority zone. Faced with this specific data snapshot, A2C-LLM generated the response matrix detailed in Figure 8b. To guard our critical zone, UAVs 1 and 2 are assigned to capture E2. Considering that E3 is far away, only UAV 3 is assigned to capture E3. In addition, to ensure the safety of the critical zone, UAVs 4 and 5 are assigned to local guard tasks. Meanwhile, as E3 is found to be close to target D, UAV 13 is called in to assist in capturing E3. The remaining UAVs 8, 9, 10, and 11 each take on guard tasks for four target points, while UAV 17 remains on standby.

From the capture results shown in Figure 8c, at Step 21, E3 is successfully captured by the nearby deployed UAV 13, and E2 is successfully captured by UAV 2. In this round of tasks, all Red Team UAVs are captured by their corresponding assigned capture UAVs.

(2) Case Scenario 2: Analysis of a Failure Case

In the failure case analysis, we selected Seed 8772, which had the lowest task completion rate (only 5%) in the multi-dimensional robustness verification experiment. After analysis, the following conclusions can be drawn.

In this scenario, there are 26 guard UAVs and 20 opposing UAVs. As shown in Figure 8d, although 7 opposing UAVs are escaping toward the most critical target A, due to the limitation of the number of guard UAVs, the model only assigns UAVs 1 and 2 to E1 for group guard. When the model detects that the number of guard UAVs is insufficient, it directly allocates capture tasks in a one-to-one (per UAV) manner. In addition, since target C is approached the least frequently, the guard UAVs configured for it are directly assigned guard tasks due to the low volume of tasks. However, the Red Team UAVs in this task are randomly configured with the highest level of speed and capability. As a result, only UAV 6 successfully completes the capture of E14 at Step 47 (Figure 8e), while the other UAVs fail to complete their capture tasks. At Step 39, UAV 14, which is performing a guard task, detects an approaching opponent and switches from guard mode to capture mode (Figure 8f).

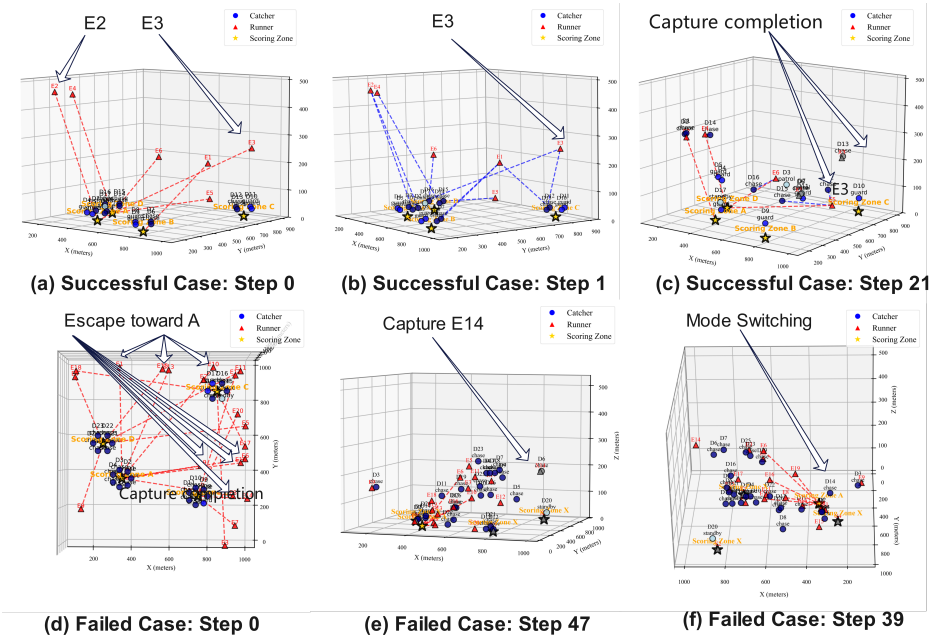


Figure 8. The 3D visual simulation of the case scenario. Blue circles denote catcher UAVs, red triangles denote Runner UAVs, and yellow stars denote scoring zones.

6.9. Engineering Feasibility Analysis

We evaluate the deployment feasibility of A2C-LLM from three dimensions: inference latency, training cost, and model footprint. End-to-end latency benchmarks are conducted across four scenario scales (4v3, 8v6, 12v7, 24v18), with five random seeds per scale and two GPU warm-up runs, on the same hardware configuration as in Section 6.1. Three standard inference-time optimizations are applied to the LLM—LoRA weight merging, output length limiting, and greedy decoding—which collectively increase generation throughput from approximately 22 tok/s to a maximum of approximately 65 tok/s without requiring model retraining. The Hungarian algorithm and PURE-A2C are evaluated under identical conditions for comparison. Results are presented in Table 5.

Table 5. End-to-end inference latency and one-shot success rate across scenario scales.

Method	Scale	T_{solve} (s)	T_{parse} (ms)	T_{total} (s)	One-Shot
A2C-LLM	4v3	11.7 ± 0.2	18.2 ± 1.1	11.7 ± 0.2	100%
	8v6	15.7 ± 0.8	28.7 ± 5.2	15.8 ± 0.8	100%
	12v7	19.0 ± 0.6	30.6 ± 1.9	19.1 ± 0.6	100%
	24v18	18.4 ± 3.8	55.4 ± 2.0	18.5 ± 3.8	100%
Hungarian	4v3	0.002	10.4	0.013	100%
	24v18	0.009	37.5	0.046	100%
PURE-A2C	4v3	0.012	12.2	0.024	100%
	24v18	0.012	34.4	0.047	100%

Note: Intermediate-scale results for Hungarian and PURE-A2C exhibit consistent trends; only the smallest and largest scales are shown for brevity.

Table 5 reveals that the parsing overhead of A2C-LLM remains at only 18–55 ms across all scales, on the same order of magnitude as Hungarian and PURE-A2C, demonstrating that the constrained decoding and feasibility projection pipeline does not constitute a meaningful computational bottleneck. The total latency of A2C-LLM is dominated entirely by LLM autoregressive generation, which accounts for over 99.8% of the total latency, with the optimized total latency ranging from 11.7 to 19.1 s. At the 24v18 scale, the latency is 18.5 s, slightly lower than 19.1 s at 12v7, and the variance is substantially reduced,

owing to greedy decoding eliminating the long-tail output phenomenon observed under sampling. Regarding training cost, complete training requires 1600 episodes. The single-step MDP formulation is essential to training feasibility: a multi-step temporal formulation would cause per-episode training time to scale linearly with the number of simulation steps, imposing prohibitive computational cost for LLM-based reinforcement learning requiring thousands of iterations. Under the current single-step design, total training time is approximately 78 h. In terms of model footprint, the LLM backbone occupies approximately 4 GB of GPU memory during FP16 inference. Hungarian and PURE-A2C achieve latency below 50 ms, but their task completion rates and perfect-win rates are substantially inferior to A2C-LLM (see Section 6.2), illustrating a deliberate performance-versus-latency trade-off at the macro-allocation level.

It is worth noting that all latency measurements were obtained on a single consumer-grade desktop equipped with an NVIDIA GPU offering 504 GB/s memory bandwidth, as detailed in Table 2. On this hardware, the 1.5B-parameter model achieves a single inference latency of 11.7–19.1 s across all tested scales, while the parsing pipeline contributes only 18–55 ms. This confirms that the inference cost is dominated by autoregressive generation rather than constraint enforcement. Three standard inference-time optimizations—LoRA weight merging, output length limiting, and greedy decoding—collectively increase generation throughput from approximately 22 tok/s to a maximum of 65 tok/s without retraining, demonstrating that substantial headroom exists even within the current hardware envelope.

Theoretically, the dominant latency factor is the memory bandwidth available for reading model weights during token generation. The present hardware provides 504 GB/s; upgrading to a device with 1555 GB/s or higher memory bandwidth would proportionally reduce the weight-transfer time. Combined with post-training quantization to reduce model size and thus the total data volume per token, and dedicated inference engines that optimize memory access patterns, the end-to-end inference latency could be compressed to an estimated 7 s or below. This estimate is conservative in that it accounts only for bandwidth scaling and quantization effects, without assuming architectural changes to the model or the inference pipeline.

In summary, the current latency figures reflect the hardware configuration under which they were measured, and multiple avenues exist for further reduction—hardware upgrades to increase memory bandwidth, post-training quantization to reduce model size and per-token data transfer volume, and dedicated inference engines to optimize memory access patterns—all of which can collectively reduce latency by several times.

6.10. Robustness Under Strategy-Level Adversary Variation

The preceding opponent capability sensitivity experiments (Section 6.4) varied the Red Team's physical parameters to alter adversarial pressure, while the multi-dimensional robustness experiments (Section 6.5) randomized scale, capability, and speed configurations. To further examine the robustness of the Blue Team when the Red Team exhibits some degree of intelligent top-level strategic variation, we introduce a Red Team strategy selector with basic situational awareness.

At the beginning of each episode, the Red Team evaluates the defensive intensity around each scoring zone and adaptively selects an attack mode accordingly, with tactical options including concentrated penetration, divided penetration with diversion, decoy and redirect, and uniform dispersion. To preliminarily simulate the perception uncertainty that may arise in real-world environments, top-level noise is randomly injected with a 5% probability—Red Team UAVs suddenly appear near the target point, and the Blue Team must complete deployment within an extremely compressed decision window. The low-level APF maneuver controller remains unchanged, with top-level target assignment

reconducted only at the start of each episode or upon the capture of the first target. All Blue Team models are evaluated via inference only, without retraining.

The experiment was conducted under the 12v7 low-capability condition, with a total of 50 episodes. Four algorithms—A2C-LLM, REINFORCE-LLM, A2C_FineTune, and Hungarian—were compared pairwise on an identical set of random seeds (consistent with Section 6.2). The results are shown in Table 6.

Table 6. Performance under strategy-level adversary variation (12v7, low capability, 50 episodes).

Method	Perfect-Win Rate (%)	Task Completion Rate (%)
A2C-LLM	52.0 [37.7, 66.3]	87.1 [81.0, 93.3]
A2C_FineTune	40.0 [25.9, 54.1]	76.0 [66.5, 85.5]
REINFORCE-LLM	26.0 [13.4, 38.6]	82.9 [78.2, 87.5]
Hungarian	10.0 [1.4, 18.6]	77.7 [74.4, 81.0]

Values are reported as mean [95% confidence interval]. Perfect-win rate denotes the fraction of episodes with 7/7 interceptions.

Under this strategy selector, A2C-LLM achieved a 52.0% perfect-win rate, outperforming the Hungarian baseline by 42.0 percentage points (Welch t-test, $p < 0.05$), with non-overlapping 95% confidence intervals. All three LLM-based algorithms significantly surpassed the Hungarian baseline in perfect-win rate. The three LLM algorithms exhibited a performance gradient of A2C-LLM > A2C_FineTune > REINFORCE-LLM, corresponding to the differences in training intensity among online RL, supervised fine-tuning, and pure REINFORCE, although the gaps among the LLM algorithms have not yet reached statistical significance (95% confidence intervals overlap).

It should be candidly noted that LLM-based algorithms introduce a 2–14% risk of catastrophic failure (i.e., episodes with zero captures), with A2C_FineTune exhibiting the highest rate (14%), A2C-LLM at 4%, and the Hungarian baseline experiencing no complete mission loss. This reveals a trade-off: LLM semantic reasoning brings a higher decision-making ceiling but also occasional unstable outputs, whereas static methods have a lower ceiling but more controllable behavioral bounds. In the current framework, constrained decoding and feasibility projection mechanisms have already reduced this risk to a relatively low level, and further improving reliability through output validation and fallback strategies represents a clear direction for improvement. Future optimization can proceed along two paths: technically, continuing to enhance the model's inference stability and decision maturity; and engineering-wise, exploring a combined solution with A2C-LLM serving as the primary policy and a static algorithm providing the fallback.

Overall, the above results demonstrate that the semantic understanding capability of A2C-LLM can sustain decision advantages when the opponent's strategic behavior pattern varies, whereas the degradation of static assignment methods is more pronounced. It should be noted that although the current Red Team strategy incorporates a situation-aware selector, its core remains rule-based. Systematic extension toward learning-based Red Team modeling has been identified as a future work direction.

7. Future Work

This work establishes a foundational framework for integrating LLMs with reinforcement learning in dynamic adversarial environments. The following directions merit further investigation:

From single-shot decision to event-triggered replanning. The current framework adopts a single macro-decision per episode, which has demonstrated excellent performance across randomized adversarial scenarios. For extreme cases involving abrupt shifts in tactical situations, the approach can be optimized to an event-triggered LLM replanning

mechanism—for instance, asynchronously triggering a new planning cycle when blue-team UAV losses exceed a threshold or when a high-value zone is breached—thereby further enhancing adaptability to severe dynamic changes. This extension will also necessitate addressing the credit assignment problem across multiple decision steps, where the current global Critic can be evolved into per-agent value functions or counterfactual baselines within a centralized-training-decentralized-execution framework.

Hierarchical and batched deployment strategy. To meet the real-time control demands of larger-scale swarms, a batched UAV dispatching architecture can be designed: the LLM generates task allocation plans for a ready-to-deploy batch, which begins execution immediately, while asynchronously triggering a new planning cycle for subsequent batches based on evolving situational conditions. This creates temporal overlap between planning and execution, thereby decoupling LLM inference latency from task execution in engineering deployments.

Inference efficiency optimization. The current 1.5B-parameter model achieves inference latency on the order of tens of seconds after standard engineering optimizations. Through model quantization, inference engine upgrades, and speculative decoding, this latency can be further compressed to the order of seconds to meet the requirements of deployment scenarios with higher real-time demands.

Red-team behavior modeling. The current red-team UAVs maneuver using artificial potential fields, without explicit opponent strategy modeling. Incorporating opponent intent recognition and belief-state reasoning can equip the LLM policy with more complete competitive reasoning capabilities. During the revision period, the preliminary exploration of a rule-based top-level strategy selector was conducted, where the Red Team adaptively chooses from tactical templates according to defensive posture. Initial results suggest that the LLM-based approach retains robustness advantages under varying opponent strategies, while systematic integration of learning-based Red Team modeling remains a direction for future work.

Validation under partial observability and noisy conditions. A preliminary close-proximity stress test—where Red Team UAVs appear within 250 m of scoring zones as a top-level equivalent of sensing uncertainty—has been conducted. The current validation is conducted under idealized simulation conditions. Verifying the robustness of the framework under sensor noise, communication delays, and partial observability is a necessary step toward practical deployment. In parallel, extending the simulation platform to incorporate higher-fidelity physical models—including 6-DoF aerodynamics, actuator constraints, and environmental disturbances—constitutes another important step toward bridging the gap between the present proof-of-concept and real-world deployment readiness.

8. Conclusions

This work proposes the A2C-LLM architecture to address the semantic-numerical gap in UAV swarm task allocation for multi-agent adversarial coordination scenarios. By integrating LLM semantic understanding with RL feedback through a lateral state-space coverage paradigm that compresses temporal reasoning into single-step contextual decisions with structured feasibility constraints, the proposed method achieves superior robustness in multi-dimensional robustness experiments, delivering a 25.7% relative improvement in perfect-win rate over the second-ranked REINFORCE-LLM and a 9.8% relative improvement in task completion rate over the second-ranked Hungarian method. Systematic experiments and ablation studies validate the effectiveness of key components, including prompt augmentation and parsing augmentation.

The key insight is that combining foundation models with principled RL training—rather than using either alone—provides a viable pathway for deploying AI systems in

reliability-critical autonomous applications. This approach addresses the fundamental tension between semantic flexibility and execution guarantees that has limited prior methods.

The current validation relies on simulation with idealized state information and simplified dynamics. Real-world deployment would introduce sensor noise, communication delays, and partial observability that are not fully captured in the present experiments. Future research directions, including those outlined in the Future Work section, aim to mature the framework for deployment in next-generation autonomous UAV systems.

Author Contributions: J.B.: conceptualization, methodology, writing—original draft, data curation. Y.Z.: funding acquisition, supervision, resources. R.Z.: writing—review and editing. P.Z.: visualization, validation, formal analysis. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Science and Technology Plan Project of Xizang Autonomous Region, grant number XZ202502ZY0061. The APC was funded by the Science and Technology Plan Project of Xizang Autonomous Region, grant number XZ202502ZY0061.

Data Availability Statement: The data presented in this study are available from the corresponding author upon reasonable request. The simulation code and trained models are not publicly available due to project restrictions.

DURC Statement: Current research is limited to the resilient cooperative control algorithms for UAV swarms, which are beneficial for civilian applications, including swarm task allocation, low-altitude delivery, and entertainment games, and do not pose a threat to public health or national security. Authors acknowledge the dual-use potential of the research involving autonomous coordination algorithms for UAV swarms and confirm that all necessary precautions have been taken to prevent potential misuse. As an ethical responsibility, authors strictly adhere to relevant national and international laws about DURC. Authors advocate for responsible deployment, ethical considerations, regulatory compliance, and transparent reporting to mitigate misuse risks and foster beneficial outcomes.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

References

1. Yang, M.; Shan, S.; Zhang, W. Decision-making and confrontation in close-range air combat based on reinforcement learning. *Chin. J. Aeronaut.* **2025**, *38*, 103526. [CrossRef]
2. Kang, H.; Joung, J.; Kim, J.; Kang, J.; Cho, Y.S. Protect Your Sky: A Survey of Counter Unmanned Aerial Vehicle Systems. *IEEE Access* **2020**, *8*, 168671–168710. [CrossRef]
3. Ou, H.; Wu, D.; Wang, S.; Wang, J.; Wang, J.; Huang, Y. The Research on the Efficiency of UAV Swarm Anti-UAV Swarm Operations. In *Man-Machine-Environment System Engineering: Proceedings of the 21st International Conference on MMESE*; Long, S., Dhillon, B.S., Eds.; Lecture Notes in Electrical Engineering; Springer: Singapore, 2022; Volume 800, pp. 341–346. [CrossRef]
4. Ariante, G.; Del Core, G. Unmanned Aircraft Systems (UASs): Current State, Emerging Technologies, and Future Trends. *Drones* **2025**, *9*, 59. [CrossRef]
5. Tian, Y.; Lin, F.; Li, Y.; Zhang, T.; Zhang, Q.; Fu, X.; Huang, J.; Dai, X.; Wang, Y.; Tian, C.; et al. UAVs Meet LLMs: Overviews and Perspectives Toward Agentic Low-Altitude Mobility. *arXiv* **2025**, arXiv:2501.02341. [CrossRef]
6. Zhao, W.; Li, L.; Zhan, H.; Wang, Y.; Fu, Y. Applying Large Language Model to a Control System for Multi-Robot Task Assignment. *Drones* **2024**, *8*, 728. [CrossRef]
7. Tagliabue, A.; Kondo, K.; Zhao, T.; Peterson, M.; Tewari, C.T.; How, J.P. REAL: Resilience and Adaptation using Large Language Models on Autonomous Aerial Robots. *arXiv* **2023**, arXiv:2311.01403. [CrossRef]
8. Cui, J.; Liu, G.; Wang, H.; Yu, Y.; Yang, J. TPML: Task Planning for Multi-UAV System with Large Language Models. In *2024 IEEE 18th International Conference on Control & Automation (ICCA)*; IEEE: Reykjavik, Iceland, 2024; pp. 886–891. [CrossRef]
9. Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; Yao, S. Reflexion: Language Agents with Verbal Reinforcement Learning. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 8634–8652.

10. Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 11809–11822.
11. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal Policy Optimization Algorithms. *arXiv* **2017**, arXiv:1707.06347. [CrossRef]
12. Wen, M.; Wan, Z.; Wang, J.; Zhang, W.; Wen, Y. Reinforcing LLM Agents via Policy Optimization with Action Decomposition. *Adv. Neural Inf. Process. Syst.* **2024**, *37*, 103774–103805.
13. Kuhn, H.W. The Hungarian method for the assignment problem. *Nav. Res. Logist.* **1955**, *2*, 83–97. [CrossRef]
14. Ahn, M.; Brohan, A.; Brown, N.; Chebotar, Y.; Cortes, O.; David, B.; Finn, C.; Fu, C.; Gopalakrishnan, K.; Hausman, K.; et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. *arXiv* **2022**, arXiv:2204.01691. [CrossRef]
15. Mnih, V.; Badia, A.P.; Mirza, M.; Graves, A.; Lillicrap, T.; Harley, T.; Silver, D.; Kavukcuoglu, K. Asynchronous Methods for Deep Reinforcement Learning. In *International Conference on Machine Learning*; PMLR: London, UK, 2016.
16. Scholak, T.; Schucher, N.; Bahdanau, D. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. *arXiv* **2021**, arXiv:2109.05093. [CrossRef]
17. Hu, E.J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv* **2021**, arXiv:2106.09685. [CrossRef]
18. Lowe, R.; Wu, Y.; Tamar, A.; Harb, J.; Abbeel, P.; Mordatch, I. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. *arXiv* **2020**, arXiv:1706.02275. [CrossRef]
19. Liu, D.; Dou, L.; Zhang, R.; Zhang, X.; Zong, Q. Multi-agent reinforcement learning-based coordinated dynamic task allocation for heterogenous uavs. *IEEE Trans. Veh. Technol.* **2023**, *72*, 4372–4383. [CrossRef]
20. Alam, M.M.; Trina, S.A.; Hossain, T.; Mahmood, S.; Ahmed, M.S.; Arafat, M.Y. Variations in Multi-Agent Actor–Critic Frameworks for Joint Optimizations in UAV Swarm Networks: Recent Evolution, Challenges, and Directions. *Drones* **2025**, *9*, 153. [CrossRef]
21. Li, G.; Lyu, Y.; Yang, D.; Jin, H.; Zhao, J.; Zhou, H. Research on the Design of Maneuver Library and Control Algorithm for Air Combat Decision-Making. In *Advances in Guidance, Navigation and Control*; Yan, L., Duan, H., Deng, Y., Eds.; Lecture Notes in Electrical Engineering; Springer Nature: Singapore, 2025; Volume 1351, pp. 86–96. [CrossRef]
22. Liu, H.; Jin, J.; Liu, K.; Zhang, J.; Niu, Y. Research on UAV Air Combat Maneuver Decision Based on Decision Tree CART Algorithm. In *Proceedings of 2022 International Conference on Autonomous Unmanned Systems (ICAUS 2022)*; Fu, W., Gu, M., Niu, Y., Eds.; Lecture Notes in Electrical Engineering; Springer Nature: Singapore, 2023; Volume 1010, pp. 2638–2650. [CrossRef]
23. Hu, K.; Xu, K.; Xia, Q.; Li, M.; Song, Z.; Song, L.; Sun, N. An overview: Attention mechanisms in multi-agent reinforcement learning. *Neurocomputing* **2024**, *598*, 128015. [CrossRef]
24. Shao, K.; Zhao, D.; Zhu, Y.; Zhang, Q. Visual Navigation with Actor-Critic Deep Reinforcement Learning. In *2018 International Joint Conference on Neural Networks (IJCNN)*; IEEE: Rio de Janeiro, Brazil, 2018; pp. 1–6. [CrossRef]
25. Barros, G.M.; Colombini, E.L. Using Soft Actor-Critic for Low-Level UAV Control. *arXiv* **2020**, arXiv:2010.02293. [CrossRef]
26. Koch, W.; Mancuso, R.; West, R.; Bestavros, A. Reinforcement Learning for UAV Attitude Control. *arXiv* **2018**, arXiv:1804.04154. [CrossRef]
27. Li, S.; Jia, Y.; Yang, F.; Qin, Q.; Gao, H.; Zhou, Y. Collaborative Decision-Making Method for Multi-UAV Based on Multiagent Reinforcement Learning. *IEEE Access* **2022**, *10*, 91385–91396. [CrossRef]
28. Wang, B.; Li, S.; Gao, X.; Xie, T. Weighted mean field reinforcement learning for large-scale UAV swarm confrontation. *Appl. Intell.* **2022**, *53*, 5274–5289. [CrossRef]
29. Nian, X.; Li, M.; Wang, H.; Gong, Y.; Xiong, H. Large-scale UAV swarm confrontation based on hierarchical attention actor-critic algorithm. *Appl. Intell.* **2024**, *54*, 3279–3294. [CrossRef]
30. Yue, L.; Yang, R.; Zuo, J.; Zhang, Y.; Li, Q.; Zhang, Y. Unmanned Aerial Vehicle Swarm Cooperative Decision-Making for SEAD Mission: A Hierarchical Multiagent Reinforcement Learning Approach. *IEEE Access* **2022**, *10*, 92177–92191. [CrossRef]
31. Chi, P.; Wei, J.; Wu, K.; Di, B.; Wang, Y. A Bio-Inspired Decision-Making Method of UAV Swarm for Attack-Defense Confrontation via Multi-Agent Reinforcement Learning. *Biomimetics* **2023**, *8*, 222. [CrossRef]
32. Wang, Z.; Zhang, J. A task allocation algorithm for a swarm of unmanned aerial vehicles based on bionic wolf pack method. *Knowl.-Based Syst.* **2022**, *250*, 109072. [CrossRef]
33. Xu, K.; Li, Y.; Sun, J.; Du, S.; Di, X.; Yang, Y.; Li, B. Targets capture by distributed active swarms via bio-inspired reinforcement learning. *Sci. China Phys. Mech. Astron.* **2025**, *68*, 218711. [CrossRef]
34. Fan, R.; Wang, J.; Han, W.; Xu, B. UAV Swarm Control Based on Hybrid Bionic Swarm Intelligence. *Guid. Navig. Control* **2023**, *3*, 2350008. [CrossRef]
35. Dong, S.; Liu, M.; Dong, S.; Zheng, R.; Wei, P. Hierarchical Heterogeneous Multi-Agent Cross-Domain Search Method Based on Deep Reinforcement Learning. *IEEE Trans. Intell. Transp. Syst.* **2024**, *25*, 18882–18898. [CrossRef]
36. Hu, B.-B.; Zhang, H.-T.; Liu, B.; Ding, J.; Xu, Y.; Luo, C.; Cao, H. Coordinated Navigation Control of Cross-Domain Unmanned Systems via Guiding Vector Fields. *IEEE Trans. Control Syst. Technol.* **2024**, *32*, 550–563. [CrossRef]

37. Mialon, G.; Dessì, R.; Lomeli, M.; Nalmpantis, C.; Pasunuru, R.; Raileanu, R.; Rozière, B.; Schick, T.; Dwivedi-Yu, J.; Celikyilmaz, A.; et al. Augmented Language Models: A Survey. *arXiv* **2023**, arXiv:2302.07842. [CrossRef]
38. Yin, S.; Fu, C.; Zhao, S.; Li, K.; Sun, X.; Xu, T.; Chen, E. A survey on multimodal large language models. *Natl. Sci. Rev.* **2024**, *11*, nwae403. [CrossRef]
39. Javaid, S.; Saeed, N.; He, B. Large Language Models for UAVs: Current State and Pathways to the Future. *arXiv* **2024**, arXiv:2405.01745. [CrossRef]
40. Chen, G.; Yu, X.; Ling, N.; Zhong, L. TypeFly: Flying Drones with Large Language Model. *arXiv* **2024**, arXiv:2312.14950. [CrossRef]
41. Zhu, F.; Huang, F.; Yu, Y.; Liu, G.; Huang, T. Task Offloading with LLM-Enhanced Multi-Agent Reinforcement Learning in UAV-Assisted Edge Computing. *Sensors* **2024**, *25*, 175. [CrossRef]
42. Xu, Y.; Jian, Z.; Zha, J.; Chen, X. Poster Abstract: Emergency Networking Using UAVs: A Reinforcement Learning Approach with Large Language Model. In *2024 23rd ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*; IEEE: Hong Kong, China, 2024; pp. 281–282. [CrossRef]
43. Zhou, Q.; Wu, J.; Zhu, M.; Zhou, Y.; Xiao, F.; Zhang, Y. LLM-QL: A LLM-Enhanced Q-Learning Approach for Scheduling Multiple Parallel Drones. *IEEE Trans. Knowl. Data Eng.* **2025**, *37*, 5393–5406. [CrossRef]
44. Murtaugh, S.A.; Criel, H.E. Fundamentals of proportional navigation. *IEEE Spectr.* **1966**, *3*, 75–85. [CrossRef]
45. Sun, J.; Tang, J.; Lao, S. Collision Avoidance for Cooperative UAVs with Optimized Artificial Potential Field Algorithm. *IEEE Access* **2017**, *5*, 18382–18390. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.